Report Spectroscopy I

Abstract

The goal of this experiment is to analyse the spectroscopy lines of hydrogene and helium and compare it with the literature values. To achive this we used a spectrometer and compared the differents angles of the grating of the spectrallines. At the end by taking the average over all result we got 3.07 ± 0.01e15, which is deviates with approxiamtely 10% of the literature value. One hypothesis is that the due to the bad resolution the mesearments are not totally correct. We conclude therefore that some deviates of the lesser visible spectrallines have to be taken in account.

1 Introduction

Spectroscopy is used to spatialy split light in its different wavelengths. Here this information is used to determine the wavelengths of the Balmer-Series. With the formula

$$\nu = R_H \left(\frac{1}{m^2} + \frac{1}{n^2} \right) \tag{1.1}$$

we caluclate the Rydberg-Frequency R_H where ν is the frequency of light and the electrone jumps from the n-th layer to the m-th layer.

All of these values are well known and the experiment is a learning excercice to get familier with spectroscopy.

2 Experiment

The experiment consists of first calibrating the spectrometer. Then to measure the H-Lines and calculate the Rydberg-Frequency R_H, the lattice-constant g, the limit of the Balmer-Series λ_{min} and the ionisation energy E_{ion} for a photon at the Rydberg-Frequency.

The equipment used is a spectrometer, He-Lamp, Hydrogen-Lamp / H-Lamp and a Laptop with ThorImageCam. The spectrometer has a dial with a measurment scale of arbitrary unit. This value will be refered to as k-value k. It turns the mirror in the spectrometer which allows to see a different wavelength range on the sensor and screen.

For calibration, the first step is to find the 0-th order. The corresponding k-value is k_0. From there the He-Fuse is connected to the spectrometer. The He-Lines all have known wavelengths. When a He-Line is found, its k-value is noted down aswell as its wavelength in Order to create a fit later. After this step a k-value is fixed. For this fixed k-value we measured the pixel position of the visible He-Lines. This is used to complete the fit-function as a function of k-value and pixel position.

Then the H-Lamp is connected. Then the k-values and pixel-position of the H-Lines are measured. Then the fit-function can be applied to the k-values and pixel posititon to find their wavelengths. From there the remaining values can be caluclated.

3 Results

k_0 denotes the k-value which corresponds to the 0-th order. The measured value is $k_0 = 950$. This value is wrong. More details are in the discussion section.

the first measurement is to fit the wavelength of the middle pixel position ($p_c = 720$) in dependance of dial position (k-value). The results are the following:

Wavelength [nm]	388.8	447.1	471.3	492.1	504.7	667.8	706.5
k-value	1310 ± 1	1131 ± 1	1101 ± 1	1072 ± 1	1070 ± 1	849 ± 1	800 ± 1

The first column contains the wavelength in nm of the measured light-strips. The value is the literature value. The second column contains the measurements. The measurement is the k-value. This is light from the He-Lamp.

the second measurement is for a fixed k-value and measures data to fit the wavelength in dependance of pixel position the results are the following:

Wavelength [nm]	492.1	501.5	504.7
pixel position	475 ± 1	802 ± 1	920 ± 1

The first column is again the known wavelengths of the measured light-strips in nm. the second column is the pixel position on the image. the image is 1440px wide, therefore the pixel position p has $0 \leq p < 1440$

the third data-set is from the Hydrogen-Lamp to measure the wavelengths of the Balmer-Series.

n	3	4	5	6
k-value	860 ± 1	1080 ± 1	1112 ± 1	1120 ± 1

the first column is the Balmer-Series. They are the corresponding measurements of the k-values.

4 Data Analysis

The fit function used is of the form $\lambda(p,k) = bk + a + \frac{p-p_{\text{cent}}}{d}$ we used this function beacause our k_0-value was wrong. Therefore we now use three fit-parameter instead of two like in the manualll. with the first data-set we obtained values for a and b. we used scipy.curve_fit for the regression. For the first data-set we used the fit $\lambda_c(k) = bk + a$ since $p = p_{\text{cent}}$. This gives the values $a = 1220 \pm 2, b = -0.663 \pm 0.002$. Here we used the the covariance matrix produced by curve_fit to determine the standart deviation for a and b. In the error propagation we used the help of AI-tools for bugfixing in the Python-Program.

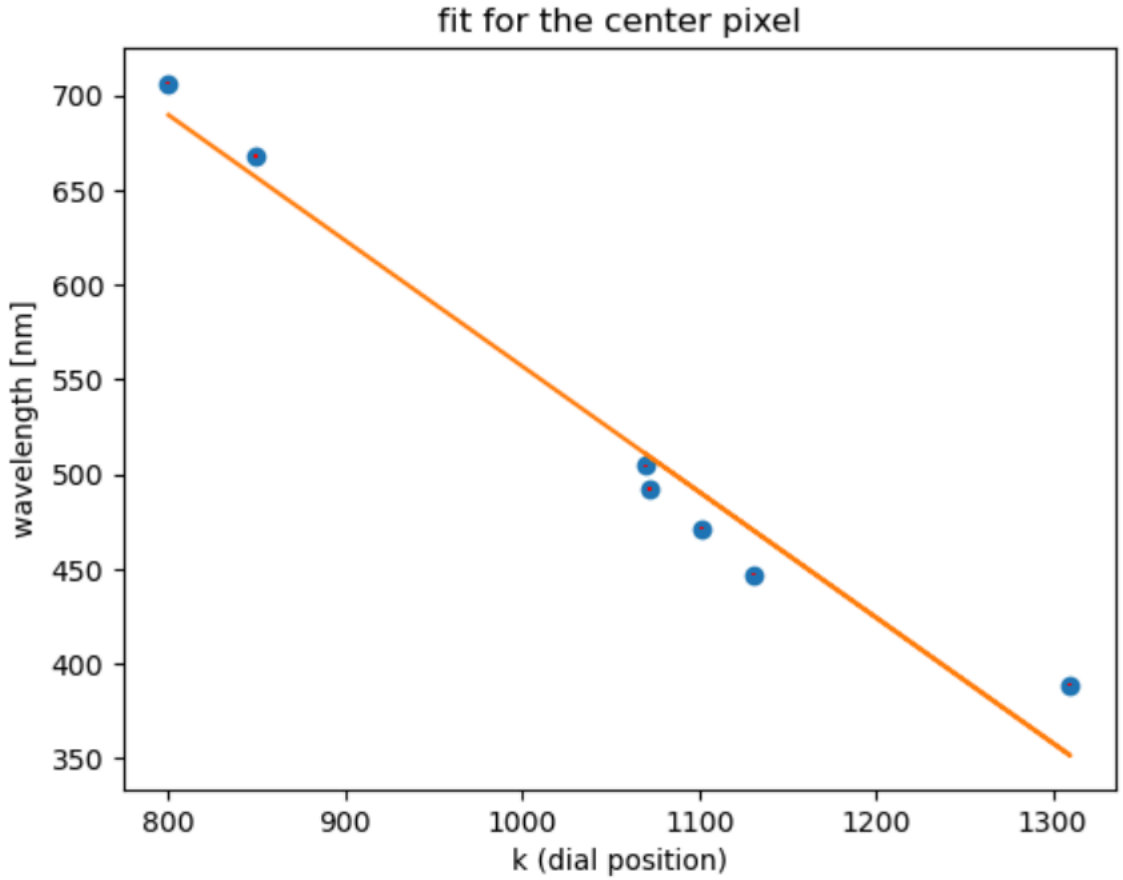


Figure 1: The Graphs shows the differents spactrallines of helium marked as points meseraude in the in the center pixel. On the x-axis we have the k-(dial position) and the y-axis is the wavelength in nanometers. The orange line is the least sqaures approximation of the points.

For the second data-set the fit function is $\lambda(p, k_{\text{fix}}) = bk_{\text{fix}} + a + \frac{p - p_{\text{cent}}}{d}$. this is the final fit function but with a fixed $k = k_{\text{fix}}$ and a, b from above. this gives $d = 42.6 \pm 0.1$. Again we used the help of AI-tools to help debug our Python-Program.

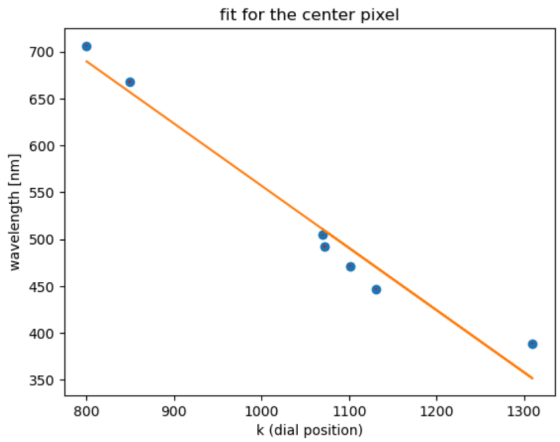


Figure 2: Screenshot of spectral range at k = 1066. We can see three different spectrallines. The verdical line marks the center pixel.

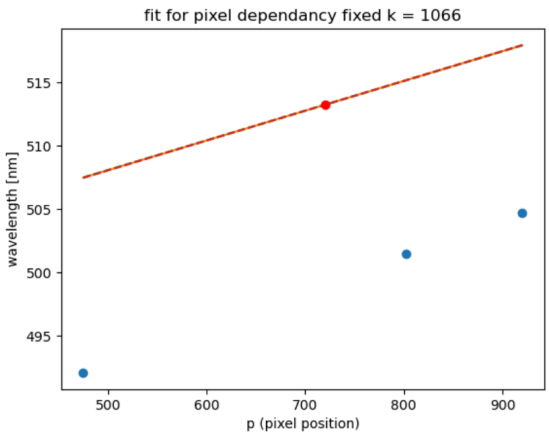


Figure 3: Graph of the pixel presentation of the visibel spectralines in the picture beside. The blue dots represents the spectrallines of the picture. The red dot is the fit of the center pixel fixed at k=1066. The red striped line represents the fit from Figure 1.

From the fit we calculate the lattice constant g as $g = \frac{1}{d} = (5.87 \pm 0.02) \cdot 10^{-5}$.

With the complete fit function we obtained the following wavelengths of the Balmer-Series through the fit function: $\lambda_3 = 650 \pm 2$ nm, $\lambda_4 = 504 \pm 2$ nm, $\lambda_5 = 483 \pm 2$ nm, $\lambda_{6_} 477 \pm 2$ nm.

From the average of the obtained wavelengths we calculate the Rydberg-Frequency as $R_{\mathrm{H}} = (3.07 \pm 0.01)e15$ therefore the limit of the Balmer-Series is $\lambda_{\mathrm{min}} = 391 \pm 2$ nm. From the Rydberg-Frequency we continue to calculate the ionisation energy as $E_{\mathrm{ion}} = \frac{hR_{\mathrm{H}}}{e} = 12.7 \pm 0.1$ eV where h is Plancks constant and e is the proton charge.

To determine the standart deviations for the obtainded values. We used Gaussian-Error-Propagation. We used AI-tools to help depug our Python-Program.

5 Discussion

The k_0-value measured in the experiment is $k_0 = 950$. This measurment does not correspond to the actual k_0-vlaue but to the 587.5 nm wavelength light. This wavelength is yellow and was therefore misinterpreted as the 0-th order. In order to correct for this mistake, we used for the first fit-function $\lambda_{\mathrm{c}}\ (k) = bk + a$ instead of $\lambda_{\mathrm{c}}\ (k) = b(k - k_0)$. The latter function is used in the manual.

The measurements of the Balmer-Series have given:

literature value [nm]	656	486	434	410
experimental value [nm]	650 ± 2	504 ± 2	483 ± 2	477 ± 2

Also the experimental values from the Rydberg-Frequency, ionisation energy and the limit of the Balmer-Series deviate from the literature values.

	Rydberg-Frequency [Hz]	ionisation energy [eV]	limit of Series [nm]
literature value	3.29 e 15	13.6	365
experimental value	$(3.07 \pm 0.01)e15$	12.7 ± 0.1	391 ± 2

The expected error traces back to the originally set values of $\Delta k = \pm 1$ and $\Delta p = \pm 1$ where Δk is the error of the k-value and Δp the error in pixel position. However they do not account for the actual error. We therefore assume that there is some systematic error. As a main source we see errors in the measurements. This could have happened at different stages. Firstly in the calibration step with the He-Lamp. A possible mistake is a misidentification of a He-Line with another He-Line and therefore a wrong data-point of the fit-function. Another possible mistake is a wrong k-value. It is possible that some readings deviate by 100 beacause we found that the disply of the k-value was difficult to read for k-values near to a full hundreth. Also it is possible that there was a copying mistake whilst writing the values down. This all would lead to errors that propagate into the fit-function. Since our data is visibly not linear we belive that one such mistake happend.

The error in the fit-function is also visible in figure 2. The first fit gives a fixed point for $p = p_c = 720$ and the free parameter varies the slope. However all data-points are below the fixed-point, likely beacause the first fit is already wrong by some offset at this position. This could lead to a wrong fit-value, since the second fit tries to compensate for the offset produced by the first fit.

Further mistakes could be wrong k-value or pixel position readings for the H-Lines, which could also contribute to the error.

6 Conclusion

In this experiment we calibrated a spectrometer and measured the H-Lines of a H-Lamp. These results were used to calculate Rydberg-Frequency R_H, the lattice-constant g, the limit of the Balmer-Series λ_{min} and the ionisation energy E_{ion} for a photon at the Rydberg-Frequency.

The experimental values of R_H, λ_{min} and E_{ion}, lie within 10% of the literature values. The error is not covered by the standart deviation of the obtained values. Since the data fit data differs from the linear fit

model, we concluded that there is systematic error in the data. We see misidentification of He-Lines in the calibration step and wrong read k-values as the main cause.

So könnte man sourcen zitieren
[1], [2] Ich mache eine Footnote[1]

[1] 1 2 3

7 Appendix

```
import numpy as np
import matplotlib.pyplot as plt

# %%
p_cent = 720                                                    # image is 1440 wide
k0 = 950                                                        # +-1 measured value
k_cent = np.array([1070, 1072, 1101, 1131, 1310, 849, 800]) # +-1 measured values of k
lamda_cent = np.array([504.7, 492.1, 471.3, 447.1, 388.8, 667.8, 706.5])
                                                                # corresponding wavelength
values from manual

# %%
# fit model for wavelength depending on dial position k
def lamda_cent_fit (k, bb, aa):
    return bb * k + aa

# %%
std_k = 50
std_k_cent = np.ones(7) * std_k # uncertenties of k values

from scipy.optimize import curve_fit
# first fit to convert x-error to y-error
popt, pcov = curve_fit(lamda_cent_fit, k_cent, lamda_cent, absolute_sigma=True)

b_prov = popt[0]

# y-error
std_lamda_from_k = np.abs(b_prov * std_k_cent)

# actual fit
popt, pcov = curve_fit(lamda_cent_fit, k_cent, lamda_cent, sigma=std_lamda_from_k,
absolute_sigma=True)

# fit parameters a, b for lamda(k) = b*k + a
b = popt[0]
a = popt[1]
print(a)
print(b)

print('========')

std_b = np.sqrt(pcov[0, 0])
std_a = np.sqrt(pcov[1, 1])
print(std_a)
print(std_b)

plt.plot(k_cent, lamda_cent, linestyle='', marker='o')
plt.plot(k_cent, lamda_cent_fit(k_cent, b, a))
plt.xlabel('k (dial position)')
plt.ylabel('wavelength [nm]')
plt.title('fit for the center pixel')
plt.errorbar(k_cent, lamda_cent, std_lamda_from_k, linestyle = '', color='r')

# %%
k_fix = 1066                                # +-1 measured value
```

```
p_pos = np.array([475, 802, 920])         # pixel position from image
lamda_k = np.array([492.1, 501.5, 504.7]) # values from manual (wavelength)

# %%
# fit model including pixel position
def lamda_fit (p, dd):
    return b * k_fix + a + (p - p_cent) / dd

# %%
std_p = 20

from scipy.optimize import curve_fit
# first fit to determine y-error from x-error
popt, pcov = curve_fit(lamda_fit, p_pos, lamda_k, absolute_sigma=True)

d_prov = popt[0]
std_lamda_from_p = np.abs(std_p / d_prov)

# actual fit
popt, pcov = curve_fit(lamda_fit, p_pos, lamda_k, sigma=std_lamda_from_p,
absolute_sigma=True)
d = popt[0]
print(d)
std_d = np.sqrt(pcov[0, 0])
print(std_d)

# %%
plt.plot(p_pos, lamda_k, linestyle='', marker='o')
offset = 0
plt.plot(p_pos, lamda_fit(p_pos, d) - offset)
plt.plot(p_pos, lamda_fit(p_pos, d + std_d) - offset, linestyle='--')
plt.plot(p_pos, lamda_fit(p_pos, d - std_d) - offset, linestyle='--')

plt.xlabel('p (pixel position)')
plt.ylabel('wavelength [nm]')
plt.title('fit for pixel dependancy fixed k = 1066')

plt.plot(p_cent, lamda_fit(p_cent, d) - offset, marker='o', color='r')

# %%
# final fit function
def lamda (p, k):
    return b * k + a + (p - p_cent) / d

# %%
# calculation of the lattice constant

# %%
lamda_at_center = 667.8                # wavelength of middle pixel
beta = 15 * 2 * np.pi / 360            # 15° in radians
k_pos = 850                            # +-1 dial position
alpha = (k_pos - 500) * 5.8 / 10**5    # calculates the angle alpha based on manual
print(alpha / (2 * np.pi) * 360)       # prints alpha in degrees

# %%
# lattice constant:
d = lamda_at_center / (2 * np.cos(beta) * np.sin(alpha))
print(1 / d)
```

```
# %%
# error prop:
del_alpha = 5.8 / 10**5
std_alpha = np.sqrt((del_alpha * std_k)**2)

del_g = 2 * np.cos(beta) * np.cos(alpha) / lamda_at_center
std_g = np.sqrt((del_g * std_alpha)**2)

print(std_g)

# %%
measurements = np.array([860, 1080, 1112, 1120])   # +-1 k values
balmer_lamda = lamda(p_cent, measurements)          #literature 656.279, 486.135,
434.0472, 410.1734
print(balmer_lamda)
c = 299792458

print('==============')

# error porp:
del_f_a = 1                             # = 1
del_f_b = measurements                  # = k
del_f_d = -(p_cent - p_cent) / d**2     # = -(p - p_cent) / d^2
del_f_p = 1 / d                         # = 1 / d
del_f_k = b                             # = b
std_lamda = np.sqrt((del_f_a * std_a)**2
               + (del_f_b * std_b)**2
               + (del_f_d * std_d)**2
               + (del_f_p * std_p)**2
               + (del_f_k * std_k)**2)

print(std_lamda)

# %%
# calculated values for rydborg frequency
nn = np.array([3, 4, 5, 6])                                     # balmer series
balmer_lamda_m = balmer_lamda / 10**9                           # wavelength in m
ryd_freq = c / balmer_lamda_m * 1 / ((1 / 4) - (1 / nn**2))     # formula from manual
print(ryd_freq)
print(np.mean(ryd_freq))
print(np.mean(ryd_freq) / 10**15)
print('---')
print(np.mean(ryd_freq) / c)

print('==============')

# error prop:
del_ryd_lamda = (-1) * c / (balmer_lamda_m)**2 * 1 / ((1 / 4) - (1 / nn**2))
std_ryd = np.sqrt((del_ryd_lamda * std_lamda / 10**9)**2)
print(std_ryd)

# %%
# limit of series: f = Rh / 4 | lamda = 4 * c / Rh
print(c / np.mean(ryd_freq) * 4 * 10**9)

print('============')
# error prop:
```

```
del_lamda_lim = (-1) * 4 * c / np.mean(ryd_freq)**2
std_lamda_lim = np.sqrt((del_lamda_lim * np.mean(std_ryd))**2)
print(std_lamda_lim * 10**9)

# %%
# E_photon = h * c / lamda          (h is planks constant)
# E_photon[eV] = E_photon / e       (e is electornic charge)
e = 1.602176634 * 10**(-19)
h = 6.62607015 * 10**(-34)

print(h * np.mean(ryd_freq) / e)

print('=======')
# error prop:
del_E_photon = h / e
sig_E_photon = np.sqrt((del_E_photon * np.mean(std_ryd))**2)
print(sig_E_photon)
```

8 Sources

Bibliography

[1] W. Demtröder, *Experimentalphysik 3: Atome, Moleküle und Festkörper*. Springer Spektrum, 2018.

[2] D. J. Griffiths, *Introduction to Quantum Mechanics*, 3rd ed. Cambridge University Press, 2018.